

AYRIK MATEMATİK (MTM2622)

27.05.2021 - 10. DERS

İletişim Bilgileri

Arş. Gör. Dr. Fatih Aylıkçı

Kimya-Metalurji Fakültesi

Matematik Mühendisliği A237

02123834616 - faylikci@yildiz.edu.tr

Avesis: <https://avesis.yildiz.edu.tr/faylikci>

Konular

1. Sayılar Teorisi ve Şifreleme
2. Modüler Aritmetik, Bölünebilirlik, Asal sayılar ve EBOB
3. Kongrüans çözümleri ve uygulamaları
4. Boolean cebrine giriş , Boolean fonksiyonları
5. Boolean temsilleri, Devrelerin minimizasyonu
6. Rekürans bağıntıları ve uygulamaları
7. Graf (çizge) ve graf (çizge) modelleri, Graf terminolojisi ve Özel tipten graflar
8. Euler ve Hamilton Yolları, En Kısa Yol Problemleri
9. Planar Graflar
10. Graflarda Renklendirme
11. Ağaçlar ve uygulamaları
12. Temel Ağaçlar, Minimum Temel Ağaçlar
13. Minimum Temel Ağaçlar ile ilgili uygulamalar

Kaynak Kitaplar

1. Kenneth H. Rosen, Discrete Mathematics and Its Applications, 7th Edition McGraw-Hill Companies, Inc., 2012.
2. Ralph P. Grimaldi, Discrete and Combinatorial Mathematics: An Applied Introduction, 5th Edition, Pearson Addison Wesley, 2004.
3. Kevin Ferland, Discrete Mathematics: An Introduction to Proofs and Combinatorics, 1st Edition, Cengage Learning, 2008.
4. Richard Johnsonbaugh, Discrete Mathematics, 7th Edition, Pearson Education, 2013.

Değerlendirme

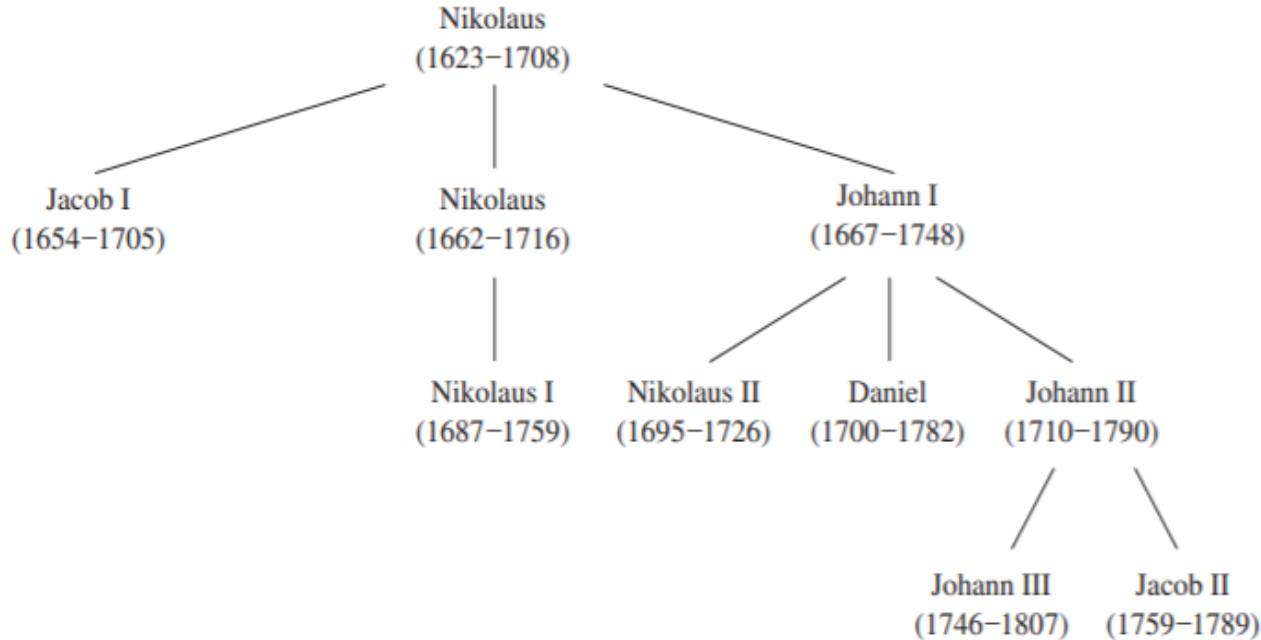
%30 1. Vize

%30 2. Vize (Sınav, Ödev veya Quiz)

%40 Final

Ağaçlar (Trees)

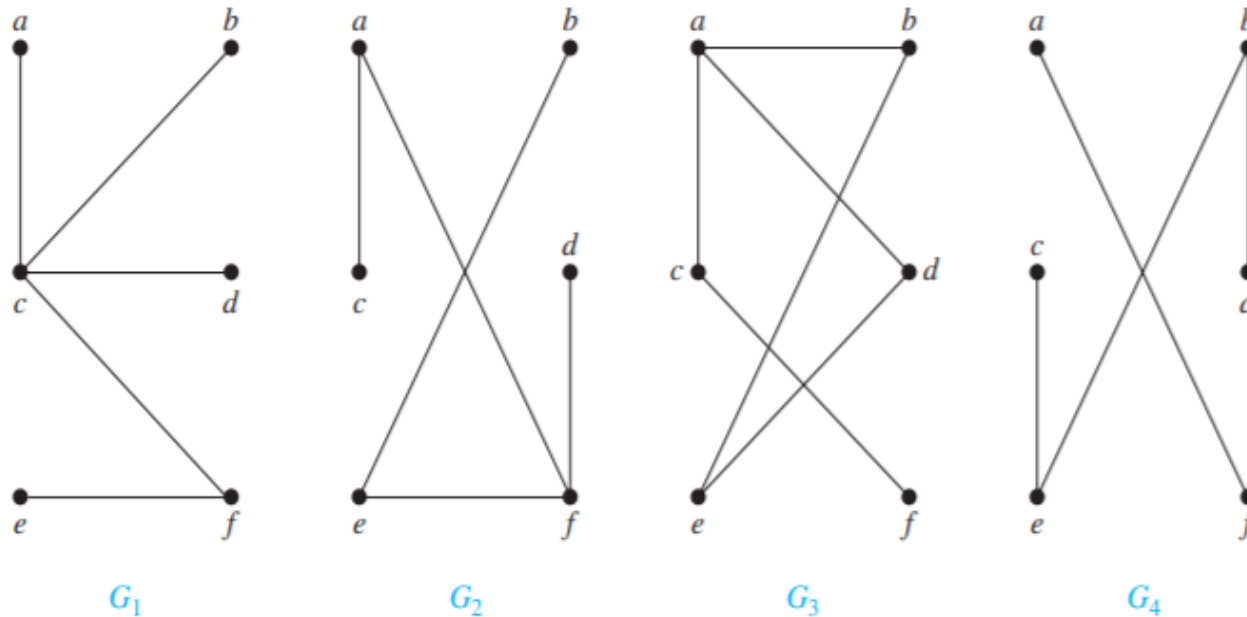
- Bu bölümde ağaçları andırdığı için ağaç ile adlandırılan belirli bir tür çizge üzerinde duracağız. Örneğin soyağaçları aile kütüğünü temsil eden çizgelerdir. Soyağacı, düğümleri ailenin fertlerini temsil etmek için kenarları da ebeveyn-çocuk ilişkisini temsil etmek için kullanır. İsviçreli matematikçilerden oluşan Bernoulli ailesinin erkek üyelerinin soyağacı şekilde gösterilmiştir. Bir soyağacını (yalnızca bir cinsiyete kısıtlanan ve akrabalar arası evlilik olmayan) temsil eden yönsüz çizge bir ağaç için örnektir.



- **Tanım 1:** Bir ağaç **hiç basit devre içermeyen** yönsüz bir çizgedir.
- Bir ağaç basit devre içermediği için, paralel kırışler veya döngü içermez. Dolayısıyla herhangi bir ağaç basit bir çizge olmak zorundadır.
- Örnek 1: Şekildeki hangi çizgeler ağaçtır?

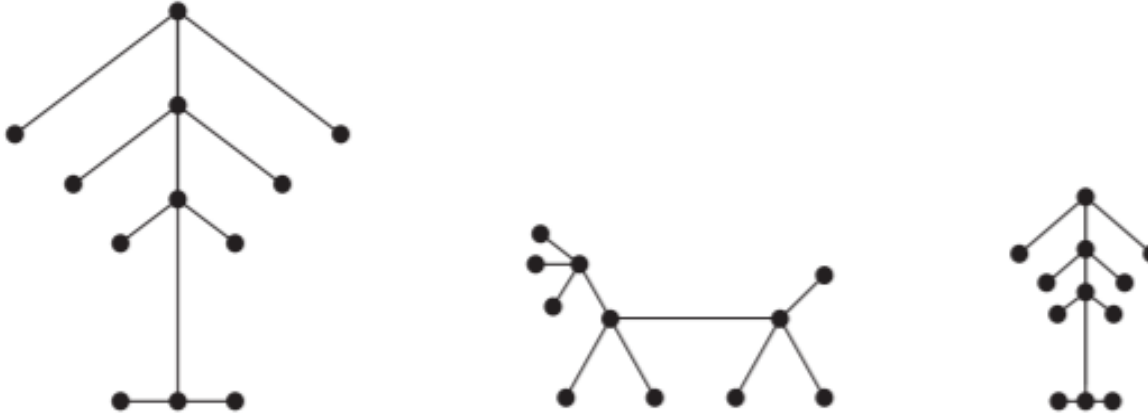
G_1 ve G_2 ağaçlardır, çünkü her ikisi de basit devre içermeyen bağlantılı çizgelerdir.

G_3 bir ağaç değildir, çünkü e, b, a, d, e bu çizge içinde olan bir basit devredir. Son olarak, G_4 de bir ağaç değildir, çünkü bağlantılı değildir.



- Hiç basit devre içermeyen bağlantılı bir çizge ağaçtır. Peki basit devre içermeyen fakat bağlantılı olmak zorunda bulunmayan çizgeler nedir? Bu çizgelere orman denir ve her bağlantılı parçaları birer ağaç olma özelliğini taşırlar. Şekilde bir orman gösterilmiştir.

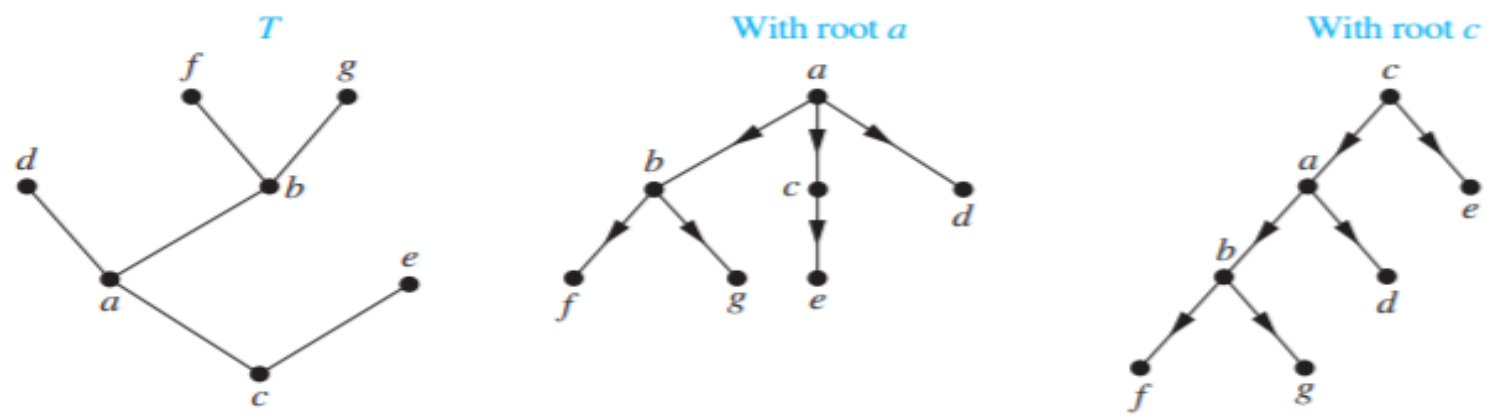
This is one graph with three connected components.



- **TEOREM 1:** Yönsüz bir çizge, ancak ve ancak her düğüm ikilisi için basit bir tek yol olması durumunda bir ağaçtır.
- **İspat:** T bir ağaç olsun. Bu durumda T basit devre içermeyen bağlantılı bir çizgedir. x ve y , T 'nin iki düğümü olsun. T bağlantılı olduğu için x ve y arasında basit bir yol vardır. Ayrıca bu yol tektir, aksi durumda ikinci bir yol olsaydı, x 'den y 'ye giden birinci yol ile ikinci yolu tersine çevirerek elde edilen y 'den x 'e giden yolu birleştirerek bir devre oluştururduk. Buradan T 'nin basit bir devreye sahip olduğunu elde ederiz. Yani bir ağacın herhangi iki düğümü arasında basit bir tek yol vardır.

Şimdi bir T çizgesinin herhangi iki düğümü arasında basit bir tek yol olduğunu kabul edelim. Herhangi iki düğüm arasında bir yol olduğu için T bağlantılıdır. Ayrıca T 'de hiç basit devre yoktur. Bunu görmek için, T 'de x ve y köşelerini içeren basit bir devre olduğunu kabul edelim. Bu durumda x ve y arasında iki yol olacaktır, çünkü verilen basit devre x 'den y 'ye giden basit bir yol ve y 'den x 'e giden basit bir yoldan oluşur. Dolayısıyla her iki düğümü arasında bir tek yol bulunduran çizge bir ağaçtır.

Köklü Ağaçlar

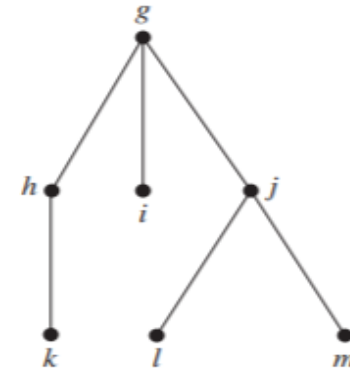
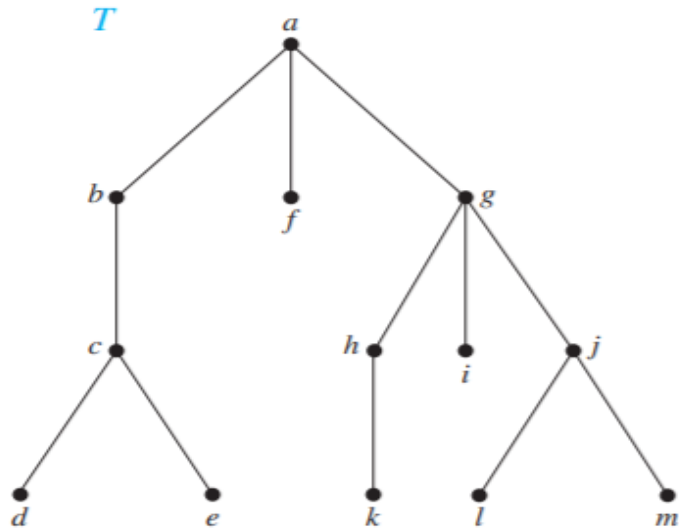


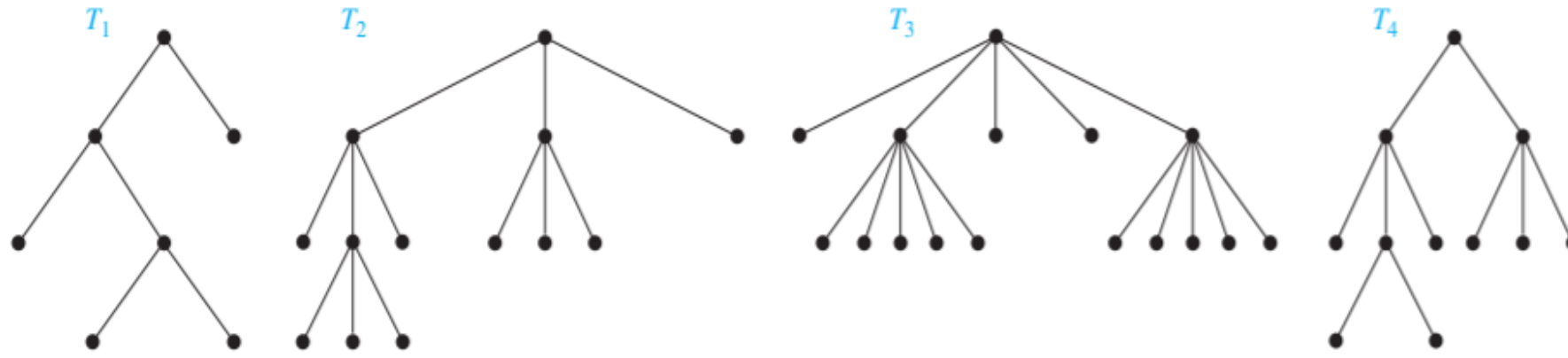
- Bir ağacın özel bir durumu kök olarak belirlenir. Bir kök belirledikten sonra kenarlara şöyle bir yön verebiliriz. Kökten her düğüme bir tek yol olduğu için (Teorem 1), yönleri kökten uzaklaşacak şekilde verebiliriz. Böylece, köküyle birlikte bir ağaç yönlü bir çizge olur ve köklü ağaç olarak isimlendirilir.
- **Tanım 2:** Bir köklü ağaç, bir düğümü kök olarak belirlenen ve her kenarı kökten uzaklaşacak şekilde yönlendirilmiş bir ağaçtır.
- Köklü ağaçlar özyineli olarak da tanımlanabilir. Köksüz bir ağacı köklü bir ağaca herhangi bir düğümü kök olarak belirleyerek değiştirebiliriz. Farklı kök seçimlerinin farklı köklü ağaçları üretebileceğine dikkatinizi çekelim. Örneğin şekilde ağaç T'den a ve c düğümleri kök seçilerek elde edilen köklü ağaçlar gösterilmektedir. Genellikle köklü ağacı, kökü çizgenin en yukarısında olacak şekilde çizeriz. Kökün seçimi kenarların yönlerini belirlediği için kenarlar üzerindeki yönü gösteren oklar ihmal edilebilir.

- Ağaçların terminolojisinin botanik ve soya ait kökeni vardır. T bir köklü ağaç olsun. Eğer v, T’de kökten farklı bir düğüm ise, v’nin ebeveyni u’dan v’ye yönlü bir kenar olacak şekildeki tek düğüm u’dur. u, v’nin ebeveyni olduğuna göre v’ye u’nun bir çocuğu denir. Aynı ebeveyne sahip düğümlere kardeş düğümler denir. Kökten verilen bir düğüme gelen yol üzerindeki verilen düğüm ve kökten farklı tüm düğümlere bu düğümün ataları denir. Bir v düğümünün nesli olan düğümler atası v olan düğümlerdir. **Çocuk düğümü olmayan düğüme yaprak düğüm denir. Çocuk düğümü olanlara ise iç düğümler denir.** Eğer kök, çizgenin tek düğümü değilse bir iç düğüm olur, diğer halde ise bir yaprak olur.
- Eğer a bir ağaçta düğümse, a düğümünün kök olduğu altağaç, a ve neslini içeren ağacın alt çizgesinden ve bu nesille ilişkili kenarlarından oluşur.

- Örnek 2: Şekildeki T (kökü a olan) ağacında c'nin ebeveynini, g'nin çocuklarını, h'nin kardeşlerini, e'nin tüm atalarını, b'nin tüm neslini, tüm iç düğümleri ve yaprakları bulunuz. Kökü g olan altağacı bulunuz.

c'nin ebeveyni b'dir. g'nin çocukları h,i ve j'dir. h'nin kardeşleri i ve j'dir. e'nin ataları c,b ve a'dır. b'nin nesli c,d ve e'dir. İç düğümler a,b,c,g,h ve j'dir. Yapraklar d,e,f,i,k,l ve m'dir. Kökü g olan altağaç sağdaki şekilde gösterilmiştir.





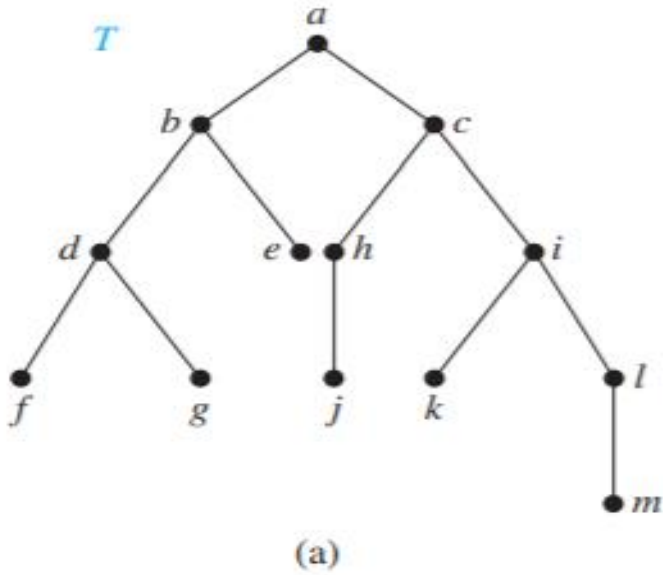
- İç düğümlerinin aynı sayıda çocuk düğümü olan köklü ağaçların birçok farklı uygulamaları vardır. Bu bölümün ilerisinde böyle ağaçları arama, sıralama ve kodlamayı içeren problemleri incelemekte kullanacağız.
- **Tanım 3:** Bir köklü ağacın her iç düğümünün m 'den fazla çocuğu yoksa bu ağaca **m -li ağaç** denir. Eğer her iç düğümünün tam olarak m tane çocuğu varsa **tam m -li ağaç** denir. $m = 2$ ise bir m -li ağaca ikili ağaç denir.
- Örnek 3: Şekilde verilen köklü ağaçlar bazı pozitif m tamsayıları için tam m -li ağaçlar mıdır?

T_1 tam ikili ağaçtır çünkü her iç düğümün iki çocuğu vardır. T_2 tam 3-lü ağaçtır çünkü her iç düğümün üç çocuğu vardır. T_3 'de her iç düğümün beş çocuğu olduğu için T_3 bir tam 5-li ağaçtır. T_4 herhangi bir m için bir tam m -li ağaç değildir, çünkü bazı iç düğümlerinin iki çocuğu varken diğerlerinin üç çocuğu vardır.

- **Sıralı Köklü Ağaçlar:** Bir sıralı köklü ağaç, her iç düğümün çocuklarının sıralandığı köklü bir ağaçtır. Sıralı köklü ağaçlar, her iç düğümün çocukları soldan sağa doğru sıralayarak çizilir.
- Bir sıralı ikili ağaçta (genellikle sadece bir ikili ağaç denir), eğer bir iç düğümün iki çocuğu varsa, ilkinde sol çocuk ve ikincisine sağ çocuk denir. Bir düğümün sol çocuğunda kökü olan ağaca sol alt ağaç, sağ çocuğunda kökü olan ağaca ise sağ alt ağaç denir. Bazı uygulamalarda bir ikili ağacın kökünden farklı her düğümün ebeveyninin ya sağ ya da sol çocuğu olarak atanmış olduğuna dikkat ediniz. Sadece bir çocuğu olan düğümlerde bile bu yapılmaktadır.

- Örnek 4: (a)'da gösterilen ikili T ağacındaki d'nin sol ve sağ çocukları nelerdir?
c'nin sol ve sağ alt ağaçları nelerdir?

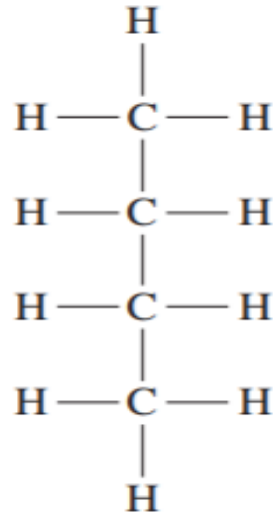
d'nin sol çocuğu f ve sağ çocuğu g'dir. c'nin sol ve sağ ağaçları sırasıyla (b) ve (c)'de gösterilmektedir.



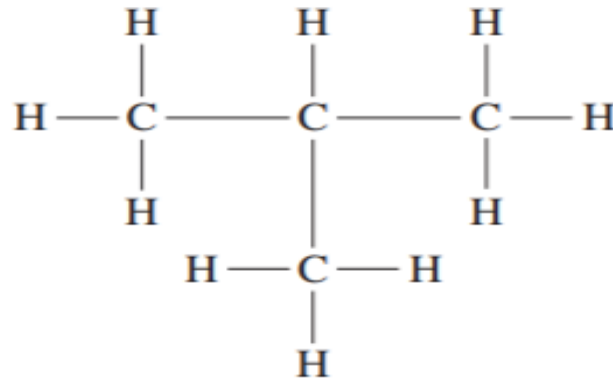
Model Olarak Ağaçlar

- ▶ Ağaçlar, bilgisayar bilimleri, kimya, jeoloji, botanik ve psikoloji gibi muhtelif alanlarda modeller olarak kullanılırlar. Ağaçlar üzerine kurulan çeşitli modellerden bahsedeceğiz.
- ▶ Örnek 5: Doymuş hidrokarbonlar ve Ağaçlar Çizgeler molekülleri göstermede kullanılabilir, atomlar düğümlerle ve aralarındaki bağlar da kenarlar ile gösterilir. İngiliz matematikçi Arthur Cayley 1857 yılında ağaçları, doymuş hidrokarbonlar olarak bilinen C_nH_{2n+2} formundaki bileşenlerin izomerlerini saymaya çalışırken keşfetmiştir.
- ▶ Doymuş hidrokarbonların çizge modelinde, her karbon atomu derecesi 4 olan bir düğümlerle ve her hidrojen atomu da derecesi 1 olan bir düğümlerle gösterilir. C_nH_{2n+2} formundaki bir bileşeni gösteren çizgede $3n+2$ tane düğüm vardır. Böyle bir çizgedeki kenar sayısı düğümlerin derecelerinin toplamının yarısı kadardır. Dolayısıyla bu çizgede $(4n+2n+2)/2=3n+1$ tane kenar vardır. Bu çizge bağlantılı ve kenar sayısı düğüm sayısından bir eksik olduğu için, bir ağaç olmak zorundadır.

- n düğümünün derecesi 4 ve $2n+2$ düğümünün derecesi 1 olan eşyapılı (izomorf) olmayan ağaçlar C_nH_{2n+2} 'nin farklı izomerlerini temsil ederler. Örneğin $n=4$ için bu tipte tam iki eşyapılı olmayan ağaç vardır. Dolayısıyla, C_4H_{10} 'un tam iki tane farklı izomeri vardır. Bunların yapıları şekilde gösterilmiştir. Bu izomerler bütan ve izobütan olarak adlandırılır.

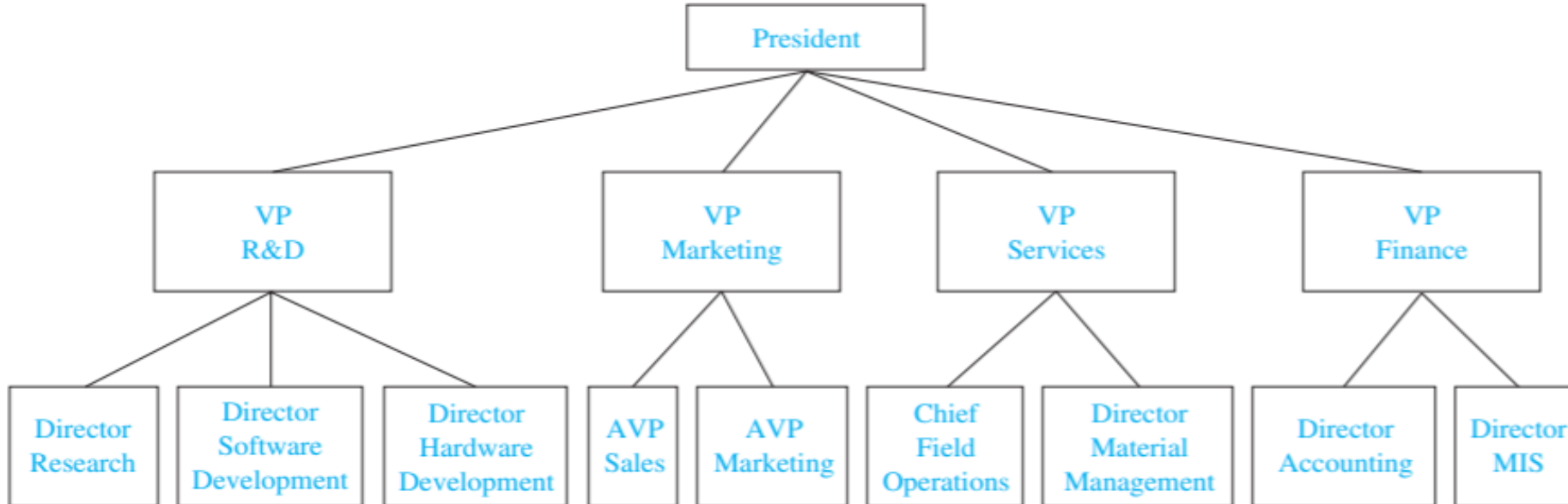


Butane

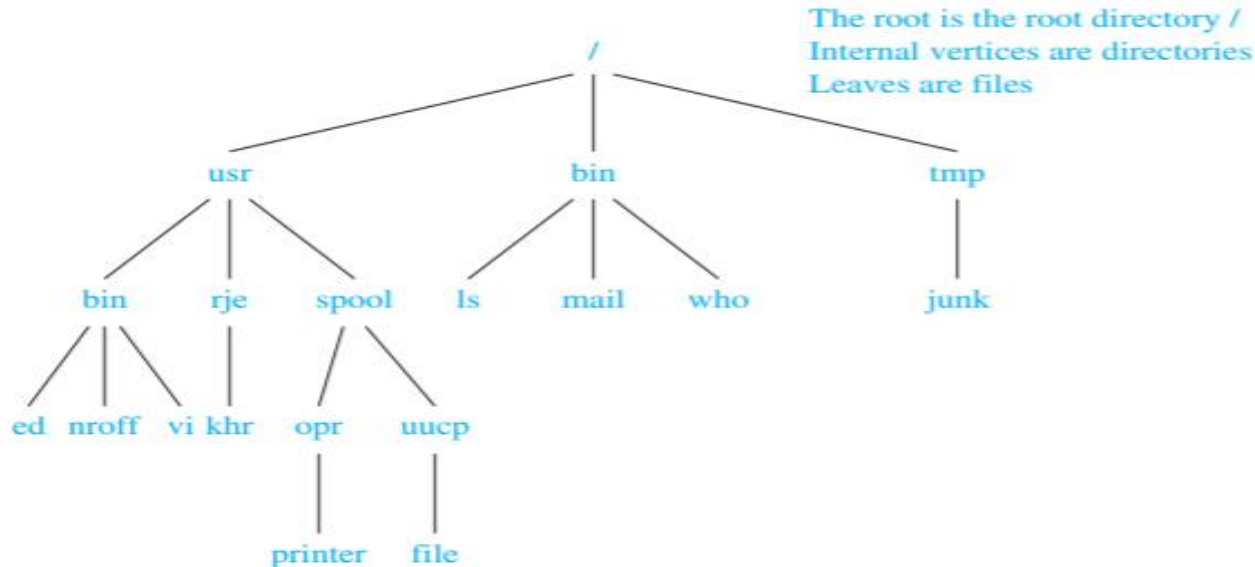


Isobutane

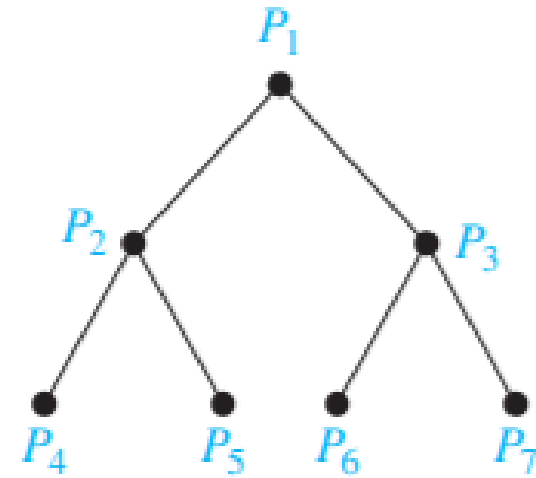
- Örnek 6: Yönetimlerin Temsilleri Büyük organizasyonların veya şirketlerin yapıları köklü ağaçlar kullanılarak modellenenebilir. Bir düğümden diğerine giden bir kenar, başlangıç düğümü ile temsil edilen kişinin son düğüm ile temsil edilen kişinin (doğrudan) patronu olduğuna işaret eder. Şekilde gösterilen çizge böyle bir ağaçtır.



- Örnek 7: Bilgisayar Dosya Sistemleri Bilgisayar hafızasında dosyalar dizinler içerisinde düzenlenebilirler. Bir dizgi hem dosyaları hem de alt dizinleri içerebilir. Ana dizgi tüm dosya sistemini içerebilir. Böylece, bir dosya sistemi bir kök ağacı ile temsil edilebilir, öyle ki kök analizini, iç düğümler alt dizinleri ve yapraklar sıradan dosyaları veya boş dizinleri temsil eder. Böyle bir dosya sistemi şekilde gösterilmiştir.



- Örnek 8: Ağaç-Bağlantılı Paralel İşlemciler Bir ağaç-bağlantılı iletişim ağı, işlemcileri birbirlerine bağlamak için bir başka önemli bir yoldur. Böyle bir iletişim ağını temsil eden çizge bir tam ikili ağaçtır. Böyle bir iletişim ağı, k pozitif bir tam sayı olmak üzere $n = 2^k - 1$ işlemciyi birbirine bağlar. Bir kök veya bir yaprak olmayan v düğümü ile temsil edilen bir işlemcinin üç tane iki-yollu bağlantısı vardır, bir tanesi v 'nin atası ile temsil edilen işlemciye giden ve iki tanesi de v 'nin iki çocuğu ile temsil edilen işlemcilere giden bağlantılardır. Kök ile temsil edilen işlemcinin, çocukları ile temsil edilen işlemcilere giden iki tane iki-yollu bağlantısı vardır. Bir v yaprağı ile temsil edilen işlemcinin, v 'nin ebeveynine giden tek iki-yollu bağlantısı vardır. Yedi işlemcisi olan bir üç-bağlantılı işletim ağı şekilde gösterilmiştir.



Ağaçların Özellikleri

- **TEOREM 2:** n düğümlü bir ağacın $n - 1$ kenarı vardır.
- **İspat: Temel Adım** $n=1$ için; $n=1$ düğümlü bir ağaçta kenar yoktur. Buradan teoremin $n=1$ için doğru olduğu çıkar.
- **Tümevarım Adımı** Tümevarım hipotezi, k bir pozitif tamsayı olmak üzere, k düğümlü her ağacın $k - 1$ kenarı olduğunu söyler. T , $k+1$ düğümlü bir ağaç, T 'nin bir yaprağı v (ağaç sonlu olduğu için varolmak zorundadır) ve w ise v 'nin ebeveyni olsun. T 'den v düğümünü ve w ile v 'yi bağlayan kenarı çıkarmak k düğümlü bir T^* ağacı oluşturur, çünkü oluşan çizge hala bağlantılıdır ve basit devresi yoktur. Buradan T 'nin k kenarı olduğu çıkar, çünkü T^* 'dan bir fazla kenarı (v ile w 'yu bağlayan kenar) vardır. Bu tümevarım adımını tamamlar.

- Bir ağacın içinde basit devre bulunmayan yönsüz bir çizge olduğunu hatırlayınız. Bu durumda, G n düğümlü bir yönsüz çizge olduğunda Teorem 2, (i) G bağlı bir çizgedir ve (ii)'de G 'de basit devre bulunmamaktadır, koşullarının (iii) G 'nin $n - 1$ kenarı vardır sonucunu bize sağladığını açıklar. Ayrıca (i)-(ii)-(iii) 'nin herhangi iki tanesi doğru olduğunda, üçüncüsü de doğru olmalı ve G bir ağaç olmalıdır.
- **Tam m -li Ağaçlarda Düğümleri Saymak** Teorem 3'te verilen bir sayı kadar iç düğümü olan bir tam m -li ağacın düğüm sayısı verilmektedir. Teorem 2'de olduğu gibi, ağacın düğüm sayısı n ile göstereceğiz.
- **TEOREM 3:** i iç düğümü olan bir tam m -li ağacın $n = mi + 1$ düğümü vardır.
- **İspat:** Kökten farklı her düğüm bir iç düğümün çocuğudur. Her i iç düğümün m çocuğu olduğu için, ağaçta kökten farklı mi düğüm vardır. Böylece ağacın $n = mi + 1$ düğümü vardır.

► T bir tam m-li ağaç olsun. Bu ağaçta iç düğüm sayısı i ve yaprak sayısı l olsun. n, i ve l 'den birisi bilindiğinde, diğer iki sayı bulunur. Teorem 4, bilinen sayıdan diğer iki sayının nasıl bulunacağını açıklamaktadır.

► **TEOREM 4:** Bir tam m-li ağaçta

(i) n düğüm varsa $i = (n - 1)/m$ iç düğüm ve $l = [(m - 1)n + 1]/m$ yaprak,

(ii) i iç düğüm varsa $n = mi + 1$ düğüm ve $l = (m - 1)i + 1$ yaprak,

(iii) l yaprak varsa $n = (ml - 1)/(m - 1)$ düğüm ve $i = (l - 1)/(m - 1)$ iç düğüm vardır.

İspat: n düğüm sayısını, i iç düğüm sayısını ve l de yaprak sayısını göstereyim. Teoremin üç kısmının hepsi Teorem 3'de verilen $n = mi + 1$ eşitliği ile $n = l + i$ eşitliğini (her düğüm ya bir yapraktır ya da bir iç düğüm olduğu için doğrudur) kullanılarak ispatlanabilir. Burada Kısım (i)'yi ispatlayacağız. Kısım (ii) ve (iii)'ün ispatları alıştırmalar olarak bırakılmıştır.

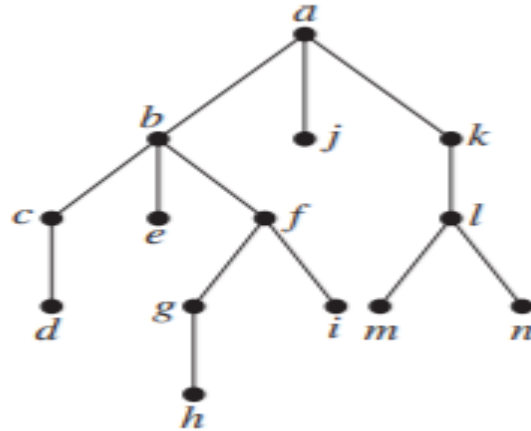
$n = mi + 1$ eşitliğinde i çözülürse $i = (n - 1)/m$ bulunur. i 'nin bu ifadesini $n = l + i$ eşitliğinde yerine yazmakla $l = n - i = n - (n - 1)/m = [(m - 1)n + 1]/m$ elde edilir.

- Örnek 9: Birisi bir zincirleme mektup başlatıyor. Her mektubu alandan mektubu dört başka kişiye iletmesi isteniyor. Bazıları bunu yapıyor, bazıları ise hiçbir mektup iletmiyor. Eğer hiç kimse birden fazla mektup almamışsa ve zincirleme mektup aldığı halde mektubu iletmeyenler 100 kişiye ulaştığında duruyorsa, ilk kişiyi de içeren kaç kişi mektubu almıştır? Kaç kişi mektubu iletmiştir?

Zincirleme mektup bir 4-lü ağaç ile temsil edilebilir. İç düğümler mektubu iletenlere ve yapraklar da mektubu iletmeyenlere karşılık gelir. 100 kişi mektubu iletmediği için bu köklü ağaçtaki yaprak sayısı $l = 100$ 'dür. Dolayısıyla, Teorem 4 (iii) mektubu görenlerin sayısının $n = (4 \cdot 100 - 1) / (4 - 1) = 133$ olduğunu söyler. Ayrıca, iç düğüm sayısı $133 - 100 = 33$ 'dür, yani 33 kişi mektubu iletmiştir.

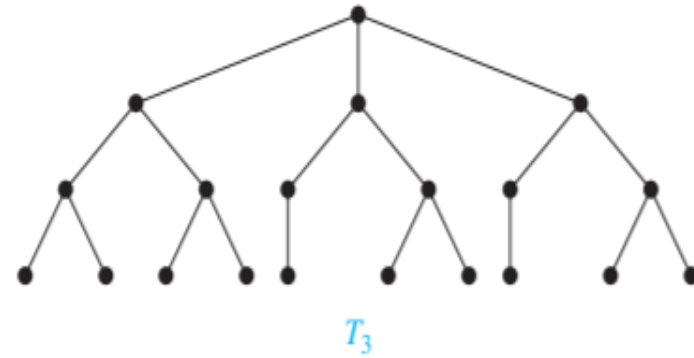
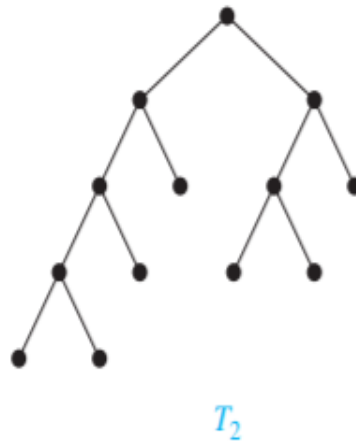
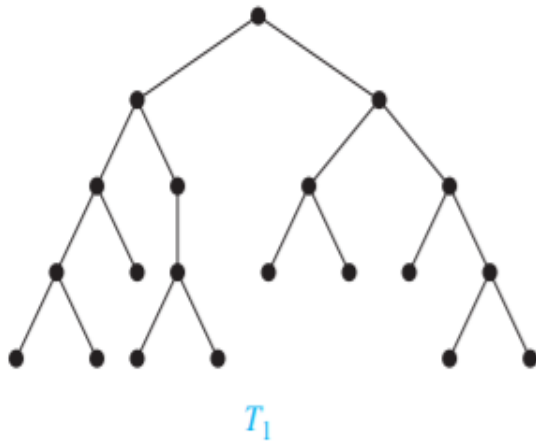
- **Dengeli m-li Ağaçlar:** Köklü ağaçlardan ‘dengeli’, yani her düğümdeki alt ağaçlara ait yollar aşağı yukarı aynı uzunlukta olanlarını kullanmak çoğu kez caziptir. Bazı tanımlar bu kavramı açıklayacaktır. Bir köklü ağaçtaki bir v düğümünün seviyesi, kökten bu düğüme giden tek yolun uzunluğudur. Kökün seviyesi sıfır olarak tanımlanır. Bir ağacın yüksekliği tüm düğümlerin seviyelerinin en büyüğüdür. Bir başka deyişle, bir köklü ağacın yüksekliği, kökten herhangi bir düğüme giden en uzun yolun uzunluğudur.
- Örnek 10: Şekilde gösterilen köklü ağacın her düğümünün seviyesini bulunuz. Bu ağacın yüksekliği nedir?

a kökünde seviye 0’dır. b, j ve k düğümlerinde seviye 1’dir. c, e, f ve l düğümlerinde seviye 2’dir. d, g, i, m ve n düğümlerinde seviye 3’tür. Son olarak, h düğümünde seviye 4’tür. En büyük seviye 4 olduğu için, bu ağacın yüksekliği 4’tür.

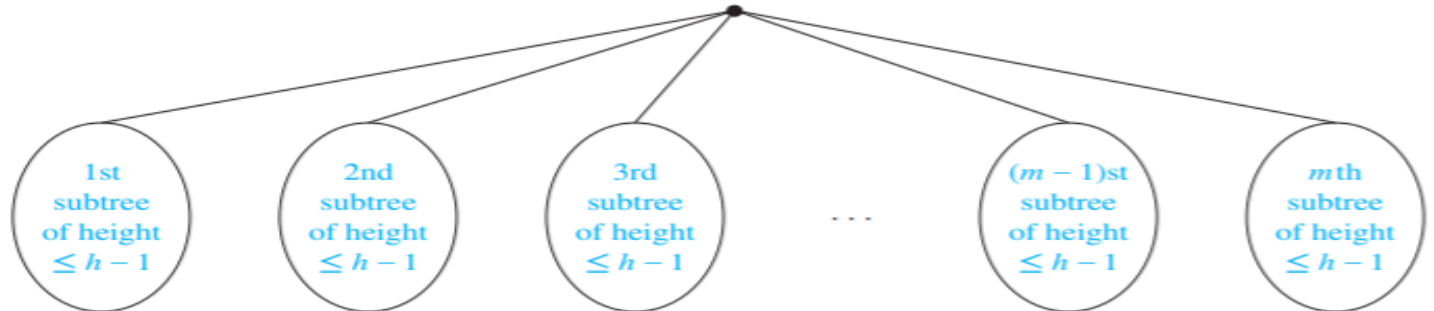


- Yüksekliği h olan bir köklü m -li ağacın tüm yapraklarının seviyesi h veya $h - 1$ ise dengelidir.
- Örnek 11: Şekilde gösterilen köklü ağaçlardan hangileri dengelidir?

T_1 dengelidir, çünkü tüm yaprakların seviyesi 3 veya 4'tür. Ancak, T_2 dengeli değildir, çünkü seviyeleri 2,3 veya 4 olan yaprakları vardır. Son olarak T_3 dengelidir, çünkü tüm yaprakların seviyesi 3'tür.



- **Bir m-li Ağaç için Yaprak Sayısının Üst Sınırı:** Genellikle m-li ağaçlarda yaprak sayısı ile ilgili bir üst sınır belirlemek yararımıza olacaktır. Teorem 5'teki sonuç m-li ağaçlarda yükseklik ile yaprak sayısını ilişkilendirir.
- **TEOREM 5:** Yüksekliği h olan bir m-li ağaçta en fazla m^h tane yaprak vardır.
- **İspat:** İspatı, yükseklik üzerine tümevarımla yapalım. İlk olarak, yüksekliği 1 olan m-li ağaçları ele alalım. Bu ağaçların her biri bir yaprak olan m'den fazla çocuğu olmayan bir kökü vardır. Bu tümevarımın temel adımıdır.
- Şimdi sonucun yüksekliği h'den küçük olan m-li ağaç için doğru olduğunu kabul edelim; bu tümevarım hipotezidir. T, yüksekliği h olan herhangi bir m-li ağaç olsun. Şekilde gösterildiği gibi, T'nin kökten seviye 1'deki düğümlere giden kenarların silinmesiyle elde edilen alt ağaçların yaprakları, T'nin yapraklarıdır.
- Bu alt ağaçların her birinin yüksekliği h - 1'den küçüktür veya eşittir. Tümevarım hipotezinden dolayı her böyle köklü ağaçların en fazla m^{h-1} tane yaprakları vardır. Böyle en fazla m^{h-1} yaprağı olan m tane alt ağaç olduğundan, bu köklü ağacın en fazla $m \cdot m^{h-1} = m^h$ yaprağı vardır. Bu da tümevarım ispatını bitirir.



- **SONUÇ 1:** Eğer yüksekliği h olan bir m -li ağacın l yaprağı varsa, $h \geq \lceil \log_m l \rceil$ 'dir. Eğer m -li ağaç tam ve dengeli ise $h = \lceil \log_m l \rceil$ 'dir. (Burada tavan fonksiyonunu kullanıyoruz. $\lceil x \rceil$ x 'den büyük veya eşit olan en küçük tamsayıyı göstermektedir.)

Ağaçların Uygulamaları

- ▶ Ağaçları kullanarak inceleyebileceğimiz üç problemi ele alacağız. Birinci problem: Bir listedeki nesneler nasıl depolanmalıdır ki kolayca yerleri bulunabilsin?
- ▶ İkinci problem: Belirli tipte nesnelerden oluşan bir koleksiyonda verilen bir özellikteki bir nesneyi bulmak için hangi karar adımları atılmalıdır?
- ▶ Üçüncü problem: Karakterlerden oluşan bir küme bit dizgileri ile verimli bir şekilde nasıl kodlanmalıdır?

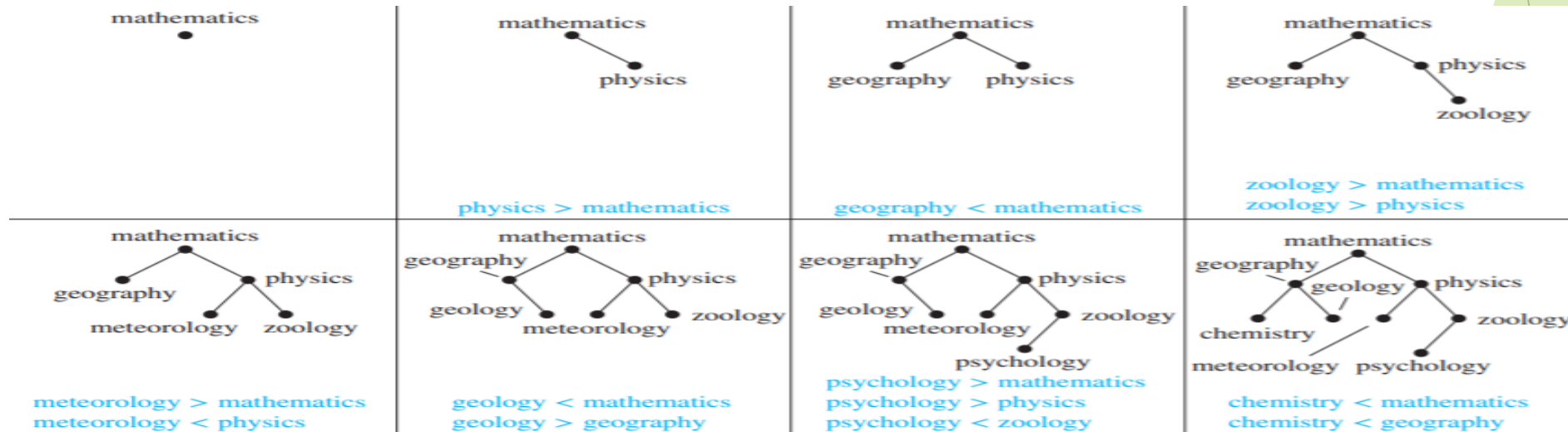
İkili Arama Ağaçları

- Bir liste içindeki öğeleri aramak bilgisayar bilimlerinde ortaya çıkan en önemli işlerden birisidir. Esas amacımız, öğeler tam sıralı olduklarında öğeleri verimli bir şekilde bulan bir arama algoritması budur. Bu amaca bir ikili arama ağacı kullanılarak ulaşılabilir. Bir ikili arama ağacı, bir düğümdeki her çocuk sağ veya sol çocuk olarak alınan, hiçbir düğümün birden fazla sağ veya sol çocuğu olmayan ve her düğüm, öğelerden birisi olan bir anahtar ile etiketlenen bir ikili ağaçtır. Ayrıca, düğümlerin anahtarları, bir düğümün anahtarı sol alt ağacının tüm düğümlerinin anahtarlarından daha büyük ve sağ alt ağacının tüm düğümlerinin anahtarlarından daha küçük olacak şekilde seçilmektedir.
- Bu özyineli prosedür, öğelerin bir listesi için bir ikili arama ağacı oluşturmak için kullanılır. Bir tane düğümü, yani kökü olan bir ağaç ile başlanır. Listedeki ilk öğe kökün anahtarı olarak seçilir. Yeni bir öğe eklemek için, ilk olarak ağaçta bulunan düğümlerin anahtarları ile karşılaştırılır, kökten başlayarak eğer gelen düğümün anahtarından daha küçük ve düğümün sol çocuğu varsa sol tarafa gidilir veya eğer gelen düğümün anahtarından daha büyük ve düğümün sağ çocuğu varsa sağ tarafa gidilir. Bu öğe gelen düğümün anahtarından küçük ve bu düğümün sol çocuğu yoksa buraya sol çocuk olarak anahtarı bu öğe olan bir düğüm eklenir. Benzer şekilde bu öğe, gelen düğümün anahtarından büyük ve bu düğümün sağ çocuğu yoksa buraya sağ çocuk olarak anahtarı bu öğe olan bir düğüm eklenir.

- Örnek 1: Matematik, fizik, coğrafya, zooloji, meteoroloji, jeoloji, psikoloji ve kimya kelimelerinin İngilizcelerini kullanarak bir ikili arama ağacı oluşturunuz.

Şekil bu ikili arama ağacını oluşturmak için gerekli aşamaları göstermektedir. Mathematics kelimesi kök olacaktır. Physics kelimesi alfabetik sırada mathematics'ten sonra geldiği için physics kelimesi köke bir sağ çocuk olarak eklenir. Geography kelimesi alfabetik sırada mathematics'ten önce geldiği için geography kelimesi köke bir sol çocuk olarak eklenir.

Bunu takiben, physics kelimesinin yer aldığı düğüme bir sağ çocuk ekleyerek buna zoology kelimesini yerleştiririz. Bunun sebebi zoology'nin mathematics'ten sonra ve physics'ten sonra gelmesidir. Benzer şekilde physics kelimesinin yer aldığı düğüme bir sol çocuk ekleyerek meteorology kelimesini yerleştiririz. Daha sonra geography kelimesinin yer aldığı düğüme bir sağ çocuk ekleyerek buna geology kelimesini yerleştiririz. Son olarak zoology kelimesinin yer aldığı düğüme bir sol çocuk ekleyerek buna psychology kelimesini yerleştiririz.



Karar Ağaçları

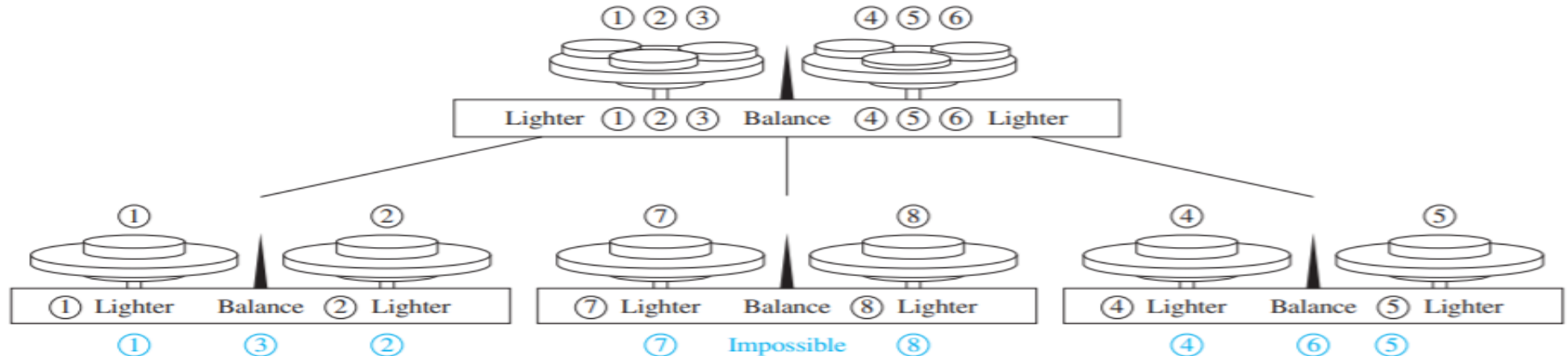
- Köklü ağaçlar, problemleri sonuca götüren kararlar serisinden oluşan modellemede kullanılabilir. Örneğin, bir ikili arama ağacı, her karşılaştırması ya öğenin yerini söyleyen ya da alt ağaçta sağa mı yoksa sola mı gidileceğini söyleyen bir karşılaştırma serisi ile öğelerin yerlerini bulmada kullanılabilir. Her iç düğümü bir karara karşılık gelen, bu düğümlerdeki alt ağaç olası karar sonuçları olan köklü bir ağaca bir karar ağacı denir. Problemin olası çözümleri bu köklü ağacın yapraklarına giden yollarına karşılık gelir.

- Örnek 3: Hepsi aynı ağırlıkta olan sekiz madeni para ve ağırlığı diğerlerinden daha az olan bir tane sahte madeni para olsun. Eşit kollu terazi kullanarak sekiz madeni paradan hangisinin sahte madeni para olduğunu belirlemek için kaç tartı yapılması gerekir? Bu sahte madeni parayı bulan bir algoritma veriniz.

Eşit kollu terazide yapılan her tartıda üç durum söz konusudur. İki taraf eşit ağırlıkta, birinci taraf daha ağır veya ikinci taraf daha ağır olabilir. Sonuç olarak, tartı serisi için oluşturulan karar ağacı 3-lü bir ağaçtır. Sekiz olası sonuç olacağı için (çünkü sekiz madeni paranın her biri hafif olan sahte madeni para olabilir) karar ağacında en az sekiz yaprak vardır ve her olası sonuç en az bir yaprak tarafından temsil edilmelidir. Sahte madeni parayı belirlemek için en fazla, karar ağacının yüksekliği kadar tartı yapmak gerekir. Karar ağacının yüksekliği en az

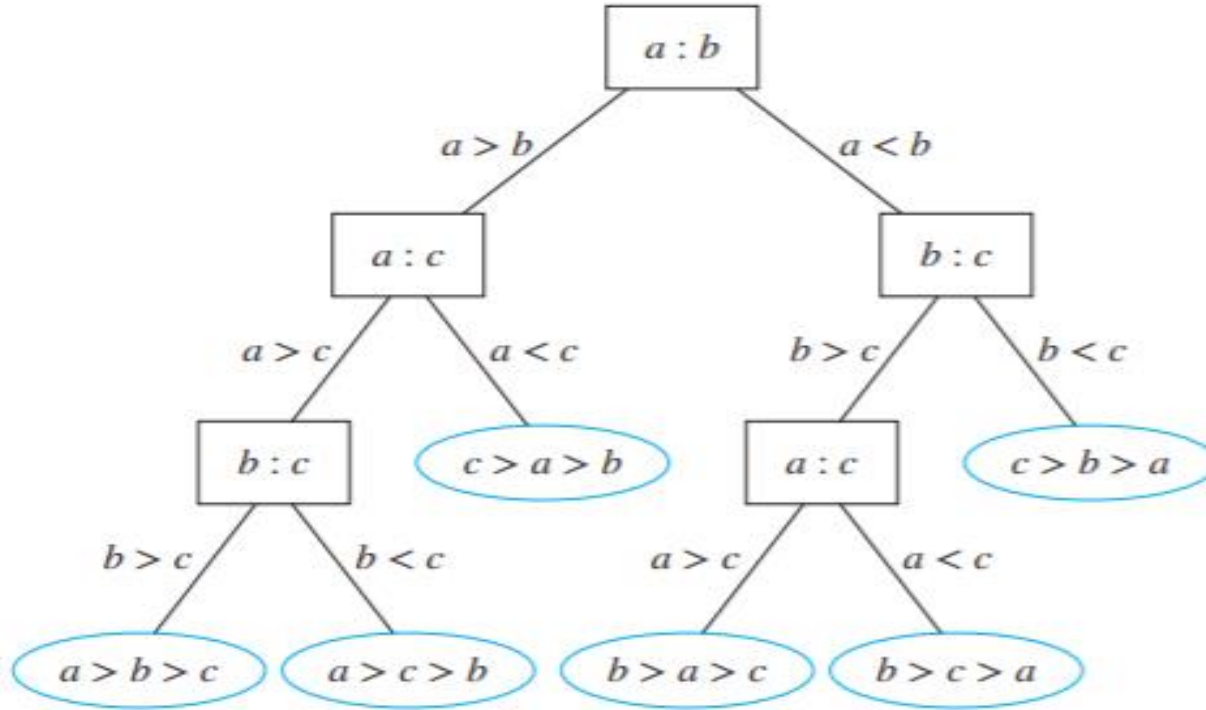
$$\lceil \log_3 8 \rceil = 2$$

'dir. Dolayısıyla en az iki tartı gerekir. İki tartı yapılarak sahte madeni parayı belirlemek mümkündür. Bunun nasıl yapıldığını anlatan karar ağacı şekilde gösterilmiştir.



- **Karşılaştırmaya dayalı sıralama algoritmalarının karmaşıklığı:** Çeşitli sıralama algoritmaları geliştirilmiştir. Belirli bir sıralama algoritmasının verimli olup olmadığına karar vermek için bu algoritmanın karmaşıklığı belirlenir. Model olarak karar ağaçları kullanarak sıralama algoritmalarının en kötü durum karmaşıklığı için bir alt sınır bulunabilir.
- Sıralama algoritmalarını modellemek için karar ağaçlarını kullanabiliriz ve bu algoritmaların en kötü durum karmaşıklığı için bir tahminde bulunabiliriz. Dikkat ediniz ki, verilen n eleman için $n!$ olası sıralama vardır. Burada çalışılan ve yaygın kullanılan sıralama algoritmaları ikili karşılaştırmalara yani bir anda iki elemanı karşılaştırmaya dayanmaktadır. Her böyle karşılaştırmaların sonucu olası sıralamaların kümesini daraltır. Böylece, ikili karşılaştırmalara dayalı bir sıralama algoritması, her iç düğümün iki elemanın bir karşılaştırmasını temsil ettiği ikili bir karar ağacı ile temsil edilebilir. Her yaprak n elemanın $n!$ permütasyonundan birisini temsil eder.

- Örnek 4: Şekilde a,b,c listesindeki elemanları sıralayan bir karar ağacını görmekteyiz.

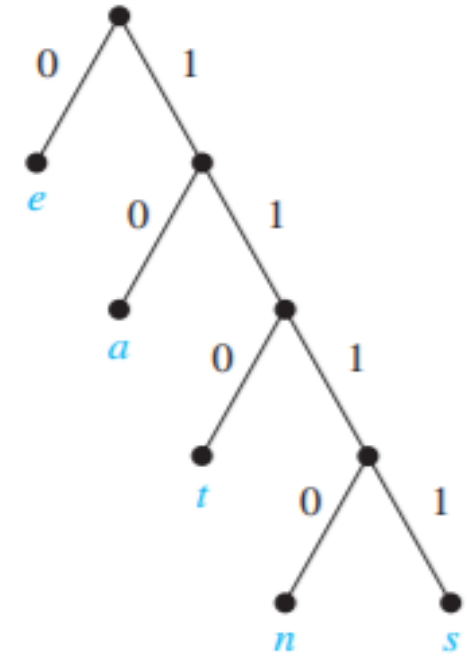


- **TEOREM 1:** İkili karşılaştırmalara dayalı bir sıralama algoritması en az $\lceil \log n! \rceil$ karşılaştırma gerektirir.

Ön Ek Kodları

- İngilizce alfabenin harflerini (küçük veya büyük harf ayrımı olmaksızın) kodlamak için bit dizgilerini (dizilerini) kullanma problemini düşünelim. Her harfi uzunluğu beş olan bir bit dizgisi ile gösterebiliriz, çünkü 26 harf ve uzunluğu beş olan 32 bit dizgisi vardır. Her karakter beş bit ile kodlandığından, bir veriyi kodlamak için metindeki karakter sayısının beş katı kadar bit kullanılır.
- Bu harfler için öyle bir kodlama yöntemi bulunabilir mi ki veri kodlandığında daha az bit kullanılsın? Eğer bu yapılabilirse hafıza kullanımından kâr edebiliriz ve iletim zamanını kısaltabiliriz.
- Harfleri kodlamada farklı uzunluktaki bit dizgileri kullanmayı düşünelim. Daha sıkça kullanılan harfler kısa bit dizgileri ile kodlanmalı ve uzun bit dizgileri de nadiren kullanılan harfleri kodlamada kullanılmalıdır. Harfler değişen büyüklükteki bitler kullanılarak kodlandığından her karakter için kullanılan bitin nerede başlayıp bittiğini belirleyen bir metod kullanılmalıdır.

- Her bir bit dizgisinin birden fazla harf dizisine karşılık gelmemesi için bir yol, bir harfe karşılık gelen bit dizgisinin bir başka harfe karşılık gelen bit dizgisinin ilk kısmı olmayacak şekilde kodlanmasıdır. Bu özellikteki kodlara ön ek kodları denir. Örneğin, e'nin 0, a'nın 10, t'nin 11 ile kodlanması bir ön ek kodlanmasıdır. Bir kelime, harfleri kodlayan tek bit dizgisinden geri bulunabilir. Örneğin, 10110 bit dizgisi ate'yi kodlamaktadır.
- Bir ön ek kodu karakterler ağacının yapraklarının etiketleri olan bir ikili ağaç ile temsil edilebilir. Ağacın kenarları, kenar sol çocuğa gidiyorsa 0 ve sağ çocuğa gidiyorsa 1 ile etiketlenir. Bir karakteri kodlamada kullanılan bit dizgisi, kökten etiketi bu karakter olan yaprağa giden tek yoldaki kenarların etiketlerinin dizisidir. Örneğin şekildeki ağaç e'nin 0, a'nın 10, t'nin 110, n'nin 1110 ve s'nin 1111 ile yapılan kodlanmayı temsil eder.

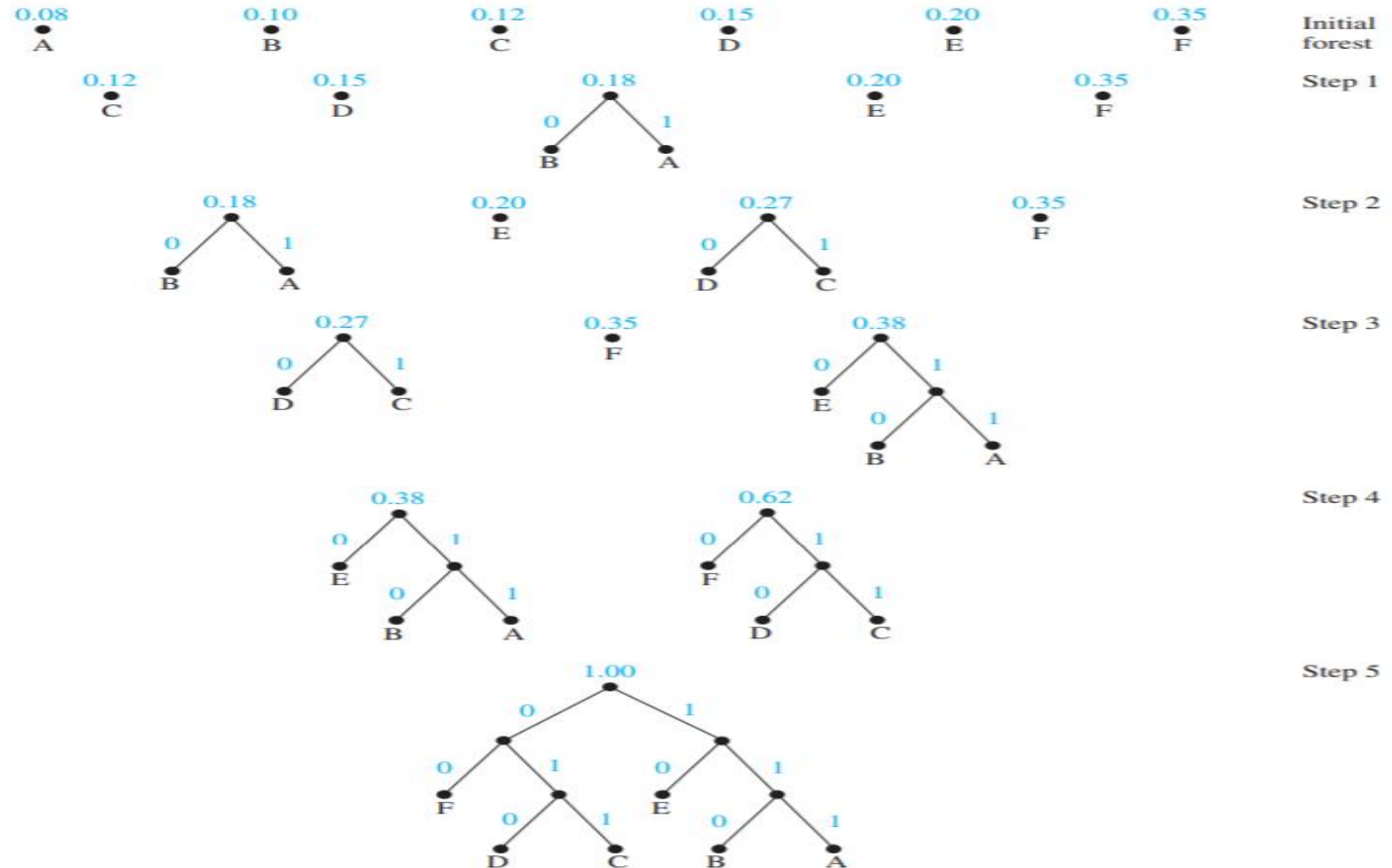


- **Huffman Kodu:** Şimdi, bir dizideki sembollerin frekanslarını (yineleme olasılıklarını) girdi olarak alan ve bu semboller için yapılabilecek tüm olası ikili ön ek kodlar içerisinde en az bit kullanan diziyi oluşturan ikili ön ek kodunu çıktı olarak verecek bir algoritma verelim. Bu algoritma Huffman kodu olarak bilinir (Bu algoritma bir dizide her sembolün kaç kez geçtiğini bildiğimizi kabul eder, dolayısıyla bir sembolün frekansını bu sembolün kaç kez geçtiğini dizinin uzunluğuna bölerek hesaplayabiliriz.)

- Örnek 5: Huffman kodunu kullanarak aşağıda frekanslarıyla birlikte listelenen sembolleri kodlayınız. A:0,08 , B:0,10 , C: 0,12 , D: 0,15 , E:0,20 , F: 0,35 . Bir karakteri kodlamak için kullanılan ortalama bit sayısı nedir?

Şekilde bu sembolleri kodlamada kullanılan adımları göstermektedir. Kodlama a'yı 111, b'yi 110, c'yi 011, d'yi 010, e'yi 10 ve f'yi 00 ile kodlar. Bu sembolü kodlamak için kullanılan ortalama bit sayısı

$$3*0,08 + 3*0,10 + 3*0,12 + 3*0,15 + 2*0,20 + 2*0,35 = 2,45$$



Oyun Ağacı

- ▶ Ağaçlar, tic-tac-toe, nim,dama ve satranç gibi bazı tipteki oyunları analiz etmek için kullanılabilir. Bu oyunların her birinde iki oyuncu sırayla hamle yaparlar. Her oyuncu diğer oyuncu tarafından yapılan hamleyi bilir ve oyunda şansın yeri yoktur. Böyle oyunları, düğümleri oyun sürecindeki konumları temsil eden, kenarları bu konumlar arasındaki geçerli hamleleri temsil eden oyun ağaçları ile modelleriz.
- ▶ Bir oyun ağacının yaprakları bir oyunun son konumlarını temsil eder. Bir yaprak tarafından temsil edilen konumda oyun bitiyorsa, bu yaprağa birinci oyuncunun kazancını gösteren bir değer veririz. Kazan-kaybet oyunlarında, birinci oyuncunun kazandığını belli etmek için çember ile temsil edilen bitiş düğümünü 1 ile etiketleriz ve ikinci oyuncunun kazandığını belli etmek için kutu ile temsil edilen bitiş düğümünü -1 ile etiketleriz. Çekilişe izin verilen oyunlarda, bir çekiliş konumuna karşılık gelen bir bitiş düğümünü 0 ile etiketleriz.

- Örnek 6: Nim Nim oyununun bir versiyonunda, bir oyunun başlangıcında belli sayıda taş yığınları vardır. İki oyuncu sırayla hamle yaparlar; yasal bir hamle, bir veya yığınların birinden geriye kalan taşların hepsini almadan birden fazla taş almaktır. Yasal bir hamle yapamayan oyuncu kaybeder. Şekilde oyun ağacı başlangıç konumunda sırasıyla iki, iki ve bir taş içeren üç taş yığını bulunan nim oyununun bu versiyonunu temsil eder. Her konumu, farklı yığınlardaki taş sayılarının bir sıralı olmayan listesi ile temsil ediyoruz. Birinci oyuncu tarafından yapılacak ilk hamle üç olası konuma götürür çünkü bu oyuncu iki taşı olan bir yığından bir taş alabilir; iki taşı olan bir yığından iki taş alabilir; veya bir taşı olan yığından bir taş alabilir. Sadece bir taşı olan bir yığın kaldığında, hiçbir yasal olan hamle mümkün değildir, yani böyle konumlar bitiş konumlarıdır. Nim bir kazan-kaybet oyunu olduğu için, bitiş düğümlerini eğer ilk oyuncunun kazandığını temsil ediyorsa +1 ile ve eğer ikinci oyuncunun kazandığını temsil ediyorsa -1 ile etiketleriz.

